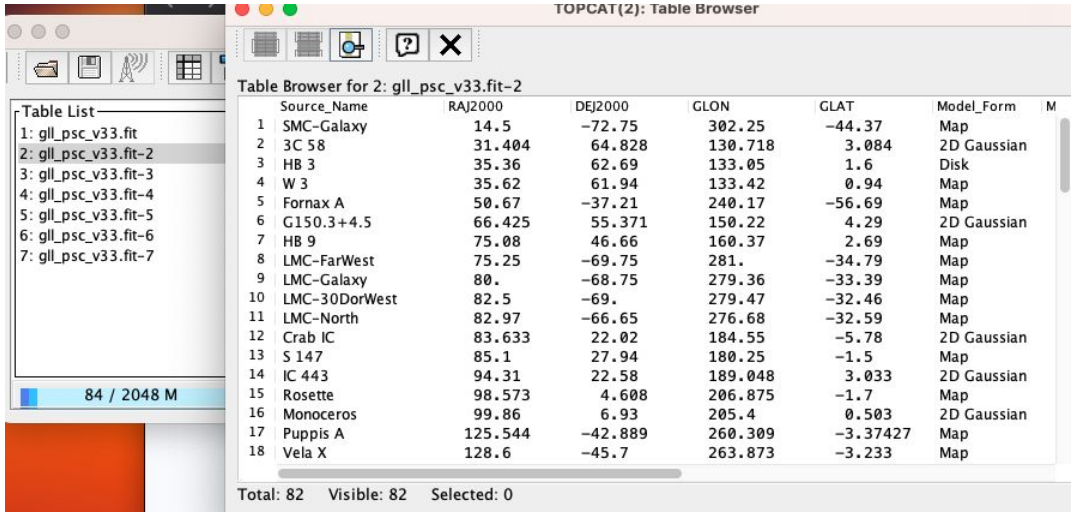
Applying ML/Deep Learning Techniques to Gamma Ray Astronomy

Krishiv Bhatia

Update: 12/08/2023

Update on Progress

- Refreshed Jerry's YouTube presentation
- Downloaded the 4FGL-DR4 FITS file from https://fermi.gsfc.nasa.gov/ssc/data/access/lat/14yr_catalog/
- Installed the TOPCAT tool on my laptop
- Viewed the 4FGL-DR4 FITS file from TOPCAT.



	Source_Name	RAJ2000	DEJ2000	GLON	GLAT	Model_Form	M
1	SMC-Galaxy	14.5	-72.75	302.25	-44.37	Map	
2	3C 58	31.404	64.828	130.718	3.084	2D Gaussian	
3	HB 3	35.36	62.69	133.05	1.6	Disk	
4	W 3	35.62	61.94	133.42	0.94	Map	
5	Fornax A	50.67	-37.21	240.17	-56.69	Map	
6	G150.3+4.5	66.425	55.371	150.22	4.29	2D Gaussian	
7	HB 9	75.08	46.66	160.37	2.69	Map	
8	LMC-FarWest	75.25	-69.75	281.	-34.79	Map	
9	LMC-Galaxy	80.	-68.75	279.36	-33.39	Map	
10	LMC-30DorWest	82.5	-69.	279.47	-32.46	Map	
11	LMC-North	82.97	-66.65	276.68	-32.59	Map	
12	Crab IC	83.633	22.02	184.55	-5.78	2D Gaussian	
13	S 147	85.1	27.94	180.25	-1.5	Map	
14	IC 443	94.31	22.58	189.048	3.033	2D Gaussian	
15	Rosette	98.573	4.608	206.875	-1.7	Map	
16	Monoceros	99.86	6.93	205.4	0.503	2D Gaussian	
17	Puppis A	125.544	-42.889	260.309	-3.37427	Map	
18	Vela X	128.6	-45.7	263.873	-3.233	Map	

Update on Progress

- Downloaded ATNF Pulsar Catalogue tar gz file. Followed README to produce the executable psrcat file.
- Followed README
- Psrcat -h works but I get error in the db_file command
- What is this for? How to fix error?

```
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % ls
README                  evalwrap.c              libpsrcat.a             psrcat.db
defineParams.c          evalwrap.o              makeit                  psrcat.h
defineParams.o          externalCall.c          myrefs.bbl              psrcat_ref
displayOutput.c         externalCall.o          parseParameters.c       readCatalogue.c
displayOutput.o         galcoord_ecliptic.c     parseParameters.o       readCatalogue.o
evaldefs.h              galcoord_ecliptic.o     plotParams.c            wc_strncmp.c
evalkern.c              galcoord_equatorial.c   plotParams.o            wc_strncmp.h
evalkern.h              galcoord_equatorial.o   psrcat                  wc_strncmp.o
evalkern.o              glitch.db               psrcat.c
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -h
-all            Merge with $PSRCAT_RUNDIR/obs*.db
-allref         Lists the entire bibliographic reference file
-bib            Provide bibliographic reference
-bibliography   Lists references within the catalogue
-c              List of parameters to obtain from the database
-check          Checks the catalogue for incorrect entries
-db_file        Database path and name
-descend        Sort in descending order
-e              Produce short ephemeris files suitable for TEMPO
-e2             Produce long ephemeris files
-e3             Produce ephemeris file with selected parameters
-exactMatch     Require exact match for string comparisons
-h              This help information
-l              Logical string for filtering unwanted pulsars
                e.g. p0 > 5
-listref        Provide bibliographic references at end of table
-merge          Merges multiple catalogue files.  For example
                -merge "mycat*.db" will merge the public catalogue
                with all the catalogues with filenames mycat*.db in the
                local directory.
-nohead         Do not print a header at the top of the table
-nonumber       Do not print line numbers in output
-nocand         Do not include pulsar candidates
-nointerim      Do not include interim solutions
-null           Null string for displaying for unknown parameter
```

```
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -db_file ./psrcat.db -c "name p0 dm" -l "p0 > 3"
zsh: illegal hardware instruction  psrcat -db_file ./psrcat.db -c "name p0 dm" -l "p0 > 3"
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar %
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -db_file ./psrcat.db
zsh: illegal hardware instruction  psrcat -db_file ./psrcat.db
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar %
```

Next Steps

- Could I have access to the Python scripts that you used with the 3FGL catalog
- Then, if I understand correctly, I need to reproduce your results using PyTorch?
- And then convert to using Neural Networks for Deep Learning?

Update: 01/08/2024

Progress

- 3FGLReproductionBasicLR Python code pointed to 3FGL Catalog file gll_psc_v16.fit.
- Updated it to point to 4FGL catalog file gll_psc_v33.fit.
- Tested with updated feature name
- Modified it for 1 hidden layer NN and DNN (2 hidden layers NN)

Results

Model	AGN Sensitivity	PSR Sensitivity	Accuracy of Best P Fit
Basic LR (0 hidden layer NN)	0.9724137931034482	1.0	0.9733333333333334
Neural Network (1 hidden layer)	0.9620689655172414	1.0	0.9633333333333334
Deep Neural Network (2 hidden layers)	0.9724137931034482	1.0	0.9733333333333334

Features Used

- Used the following features from columns in 4FGL catalog file gll_psc_v33.fit
  - ["PL_Index", "LP_Index", "PLEC_IndexS", "Variability_Index", "Unc_PL_Flux_Density", "Unc_LP_Flux_Density", "Unc_Energy_Flux100", "Unc_PLEC_Flux_Density", "Unc_Flux1000", "LP_beta", "Frac_Variability", "LP_SigCurv", "Signif_Avg"]
  - Found the corresponding 3FGL columns in the 4FGL list
  - Did some trial and error for other columns and checked model performance
- Column list is in pages 32-33 of Abdollahi_2020_ApJS_247_33 paper

Columns

- Printed out from code

1 DataRelease
2 RAJ2000
3 DEJ2000
4 GLON
5 GLAT
6 Conf_68_SemiMajor
7 Conf_68_SemiMinor
8 Conf_68_PosAng
9 Conf_95_SemiMajor
10 Conf_95_SemiMinor
11 Conf_95_PosAng
12 ROI_num
13 Extended_Source_Name
14 Signif_Avg
15 Pivot_Energy
16 Flux1000
17 Unc_Flux1000
18 Energy_Flux100
19 Unc_Energy_Flux100
20 SpectrumType
21 PL_Flux_Density
22 Unc_PL_Flux_Density
23 PL_Index
24 Unc_PL_Index
25 LP_Flux_Density
26 Unc_LP_Flux_Density
27 LP_Index
28 Unc_LP_Index
29 LP_beta
30 Unc_LP_beta
31 LP_SigCurv
32 LP_EPeak
33 Unc_LP_EPeak
34 PLEC_Flux_Density
35 Unc_PLEC_Flux_Density
36 PLEC_IndexS
37 Unc_PLEC_IndexS
38 PLEC_ExpfactorS
39 Unc_PLEC_ExpfactorS
40 PLEC_Exp_Index
41 Unc_PLEC_Exp_Index
42 PLEC_SigCurv
43 PLEC_EPeak
44 Unc_PLEC_EPeak
45 Npred
46 Flux_Band
47 Unc_Flux_Band
48 nuFnu_Band
49 Sqrt_TS_Band
50 Variability_Index
51 Frac_Variability
52 Unc_Frac_Variability
53 Signif_Peak
54 Flux_Peak
55 Unc_Flux_Peak
56 Time_Peak
57 Peak_Interval
58 Flux_History
59 Unc_Flux_History
60 Sqrt_TS_History
61 ASSOC_4FGL
62 TEVCAT_FLAG
63 ASSOC_TEV
64 CLASS1
65 ASSOC1
66 ASSOC_PROB_BAY
67 ASSOC_PROB_LR
68 RA_Counterpart
69 DEC_Counterpart
70 Unc_Counterpart
71 Flags
72 ASSOC_FGL
73 ASSOC_FHL
74 ASSOC_GAM1
75 ASSOC_GAM2
76 ASSOC_GAM3
77 CLASS2
78 ASSOC2

Feature Transformation

- FGL4Dataset feature transformations

```
def __init__(self, wanted_type_col, fluxindices, wantedtypes, wantedfeaturenames, pointsourcecata  2  4
    # Create x/input data
    xdata = []
    ydata = []
    # Select sources that are a part of the wanted types
    for source in pointsourcecatalogue.data:
        for feature_names in wantedfeaturenames:
            print("{0} = {1}".format( *args: feature_names, source[feature_names]))
        if source[wanted_type_col] in wantedtypes:
            unit = []
            # Append wanted params. Applied a log transformation to paras with highly skewed distributions.
            unit.append(float(source[wantedfeaturenames[0]]))
            unit.append(float(source[wantedfeaturenames[1]]))
            unit.append(float(source[wantedfeaturenames[2]]))
            unit.append(float(source[wantedfeaturenames[3]]))
            unit.append(math.log(source[wantedfeaturenames[4]])
                        if (source[wantedfeaturenames[4]] > 0) else float(source[wantedfeaturenames[4]]))
            unit.append(math.log(source[wantedfeaturenames[5]])
                        if (source[wantedfeaturenames[5]] > 0) else float(source[wantedfeaturenames[5]]))
            unit.append(math.log(source[wantedfeaturenames[6]])
                        if (source[wantedfeaturenames[6]] > 0) else float(source[wantedfeaturenames[6]]))
            unit.append(math.log(source[wantedfeaturenames[7]])
                        if (source[wantedfeaturenames[7]] > 0) else float(source[wantedfeaturenames[7]]))
            unit.append(math.log(source[wantedfeaturenames[8]])
                        if (source[wantedfeaturenames[8]] > 0) else float(source[wantedfeaturenames[8]]))
            unit.append(float(source[wantedfeaturenames[9]]))
            unit.append(float(source[wantedfeaturenames[10]]))
            unit.append(float(source[wantedfeaturenames[11]]))
            unit.append(float(source[wantedfeaturenames[12]]))
```

Apply log transformations to params with highly skewed distributions

```
PL_Index = 2.5443360805511475
LP_Index = 2.5455660820007324
PLEC_IndexS = 2.539679765701294
Variability_Index = 25.699934005737305
Unc_PL_Flux_Density = 3.7675866251677195e-14
Unc_LP_Flux_Density = 4.284851688957439e-14
Unc_Energy_Flux100 = 4.238916617889388e-13
Unc_PLEC_Flux_Density = 3.952874743566419e-14
Unc_Flux1000 = 2.748007377206818e-11
LP_beta = -0.009841139428317547
Frac_Variability = 0.5357248187065125
LP_SigCurv = 0.0
Signif_Avg = 8.640633583068848
```

Neural Networks Configuations Used

- Used
  - 0 hidden layer Neural Networks - Linear Regression
  - 1 hidden layer NN
  - Deep Neural Networks (2 hidden layers)

0 hidden layer NN - Linear Regression

```
Linear(in_features=13, out_features=1, bias=True)
```

NN (1 hidden layer)

```
Sequential(
  (0): Linear(in_features=13, out_features=60, bias=True)
  (1): ReLU()
  (2): Linear(in_features=60, out_features=1, bias=True)
)
```

DNN (2 hidden layers)

```
Sequential(
  (0): Linear(in_features=13, out_features=250, bias=True)
  (1): ReLU()
  (2): Linear(in_features=250, out_features=40, bias=True)
  (3): ReLU()
  (4): Linear(in_features=40, out_features=1, bias=True)
)
```

Other Changes & Github

Classes

- Separated the large code into classes for code modularity, improvement
- Created separate classes for:
  - **Driver:** 3FGLReproductionBasicLR_v1.py, 4FGLReproductionBasic{LR, NN, DNN}.py,
  - **Feature transformation:** FGL3Dataset.py / FGL4Dataset.py
  - **Models:** LogisticRegression.py, NeuralNetwork.py, DeepNeuralNetwork.py
  - **Utility functions:** utils.py (e.g. run_iteration)

- 3FGLReproductionBasicLR_v1.py
  - FGL3Dataset.py
  - LogisticRegression.py
  - utils.py

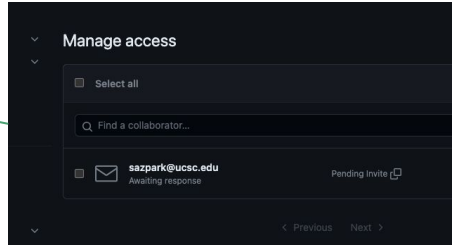
- 4FGLReproductionBasicLR.py
  - FGL4Dataset.py
  - LogisticRegression.py
  - utils.py

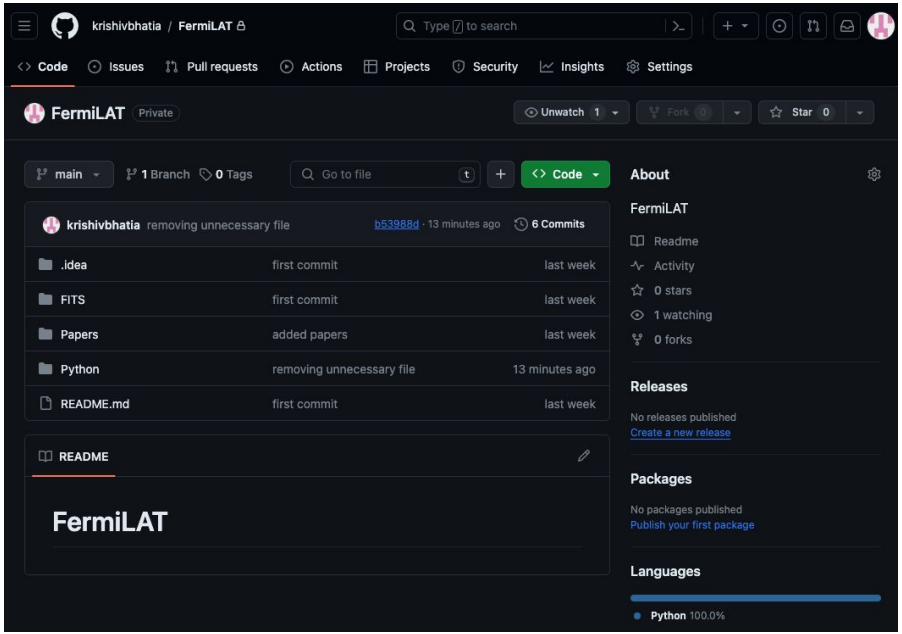
- 4FGLReproductionBasicLR.py
  - FGL4Dataset.py
  - LogisticRegression.py
  - utils.py
- 4FGLReproductionBasicDNN.py
  - FGL4Dataset.py
  - DeepNeuralNetwork.py
  - utils.py

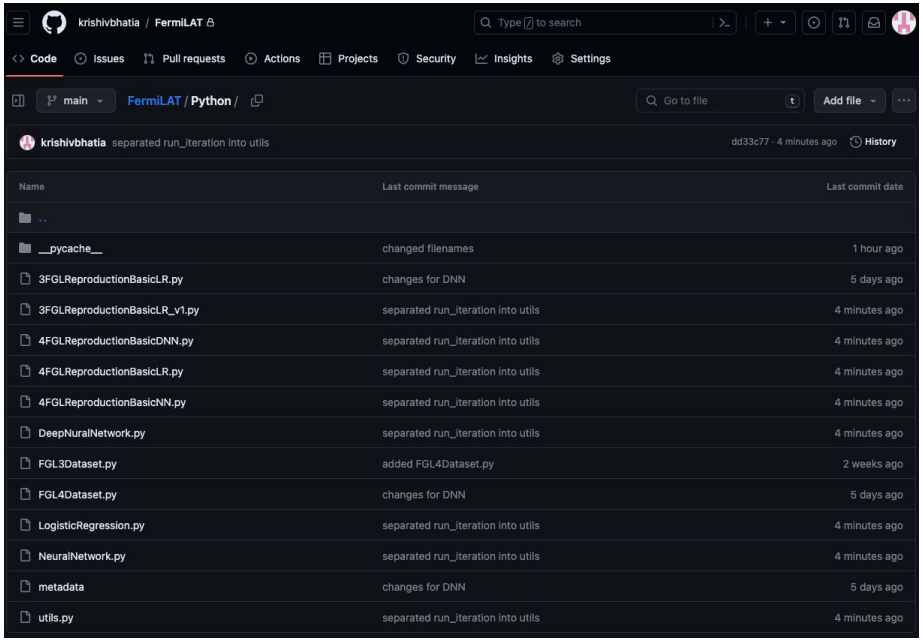
Github Repo

- Checked into github
  - https://github.com/krishivbhatia/FermiLAT/

Sent you collaborator invite





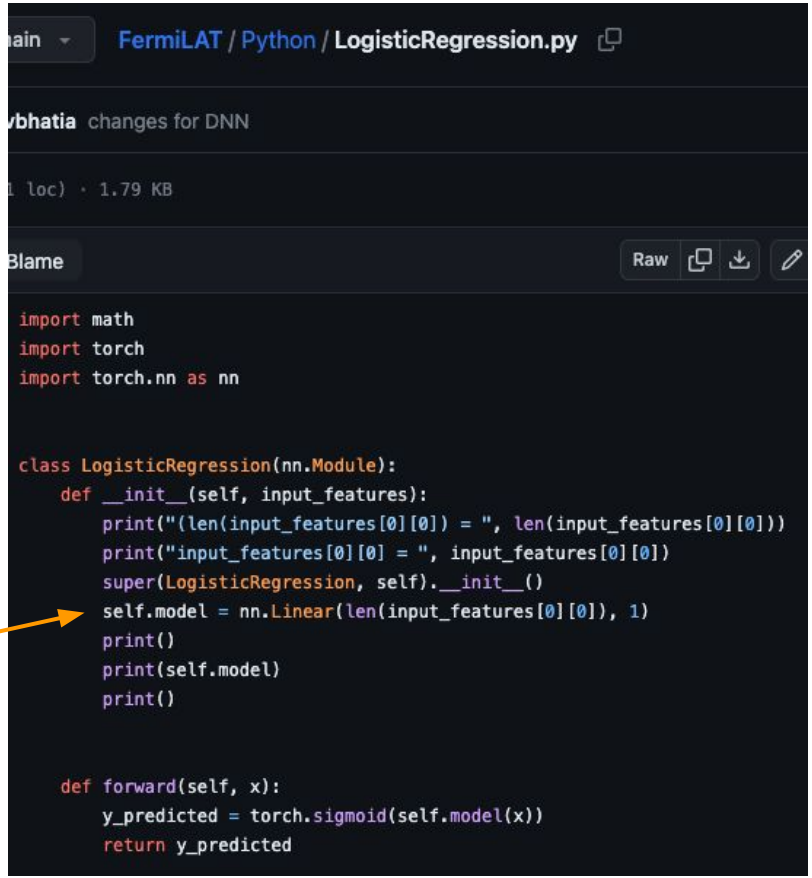


Results: Linear Regression

Files for Linear Regression

- Python Scripts
  - 4FGLReproductionBasicLR.py
  - FGL4Dataset.py
  - LogisticRegression.py
  - utils.py
- Execution
  - python3 4FGLReproductionBasicLR.py

1-layer NN (No hidden layer)
Basic LR



ain FermiLAT / Python / LogisticRegression.py

vbhatia changes for DNN

loc) · 1.79 KB

Blame Raw

```
import math
import torch
import torch.nn as nn


class LogisticRegression(nn.Module):
    def __init__(self, input_features):
        print("(len(input_features[0][0]) = ", len(input_features[0][0]))
        print("input_features[0][0] = ", input_features[0][0])
        super(LogisticRegression, self).__init__()
        self.model = nn.Linear(len(input_features[0][0]), 1)
        print()
        print(self.model)
        print()


    def forward(self, x):
        y_predicted = torch.sigmoid(self.model(x))
        return y_predicted
```

Results

- Linear(in_features=13, out_features=1, bias=True)

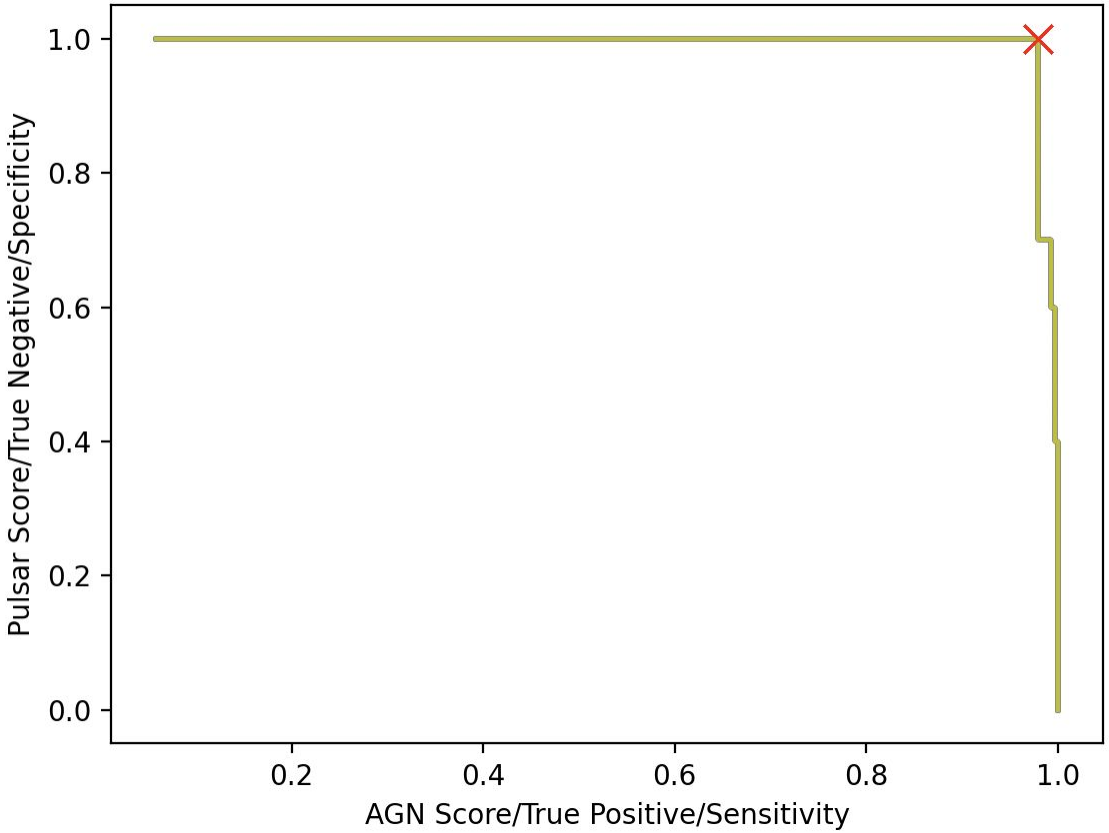
- **Results (200 iterations, lr = 0.00001)**
  - optimal_p = 0.9352
  - Best True Positive (AGN Successfully Identified): 282 (290)
  - Best AGN/True Positive/Sensitivity : 0.9724137931034482
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.9733333333333334

Results

- Linear(in_features=13, out_features=1, bias=True)

- **Results (300 iterations, lr = 0.00001)**
  - optimal_p = 0.9352
  - Best True Positive (AGN Successfully Identified): 282 (290)
  - Best AGN/True Positive/Sensitivity : 0.9724137931034482
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.9733333333333334

Plot


1.0
0.8
0.6
0.4
0.2
0.0
Pulsar Score/True Negative/Specificity
0.2
0.4
0.6
0.8
1.0
AGN Score/True Positive/Sensitivity

Results: NN - 1 hidden layer

Files for NN (1 hidden layer)

- Python Scripts
  - 4FGLReproductionBasicNN.py
  - FGL4Dataset.py
  - NeuralNetwork.py
  - utils.py
- Execution
  - python3 4FGLReproductionBasicNN.py

1 hidden layers NN
ReLU activation function

main FermiLAT / Python / NeuralNetwork.py

krishivbhatia changed model names

Code Blame 54 lines (48 loc) · 2.28 KB Code 55% faster with GitHub Copilot

```
1   import math
2   import torch
3   import torch.nn as nn
4
5
6   # Logistic Regression Model
7   #   https://www.youtube.com/watch?v=OGpQxIkR4ao&list=PLqnslRFeH2UrcDBWF5mfPGpqQDSta6VK4&index=8
8   # However I used 2 layers of neurons instead of 1 as it yields better results.
9   # 1 layers of neurons would just be 1/(1 + e^-(input1 * weight1 + input2 * weight2 + input3 * weigh
10  class NeuralNetworkModel(nn.Module):
11      def __init__(self, input_features):
12          print("(len(input_features[0][0]) = ", len(input_features[0][0]))
13          print("input_features[0][0] = ", input_features[0][0])
14          super(NeuralNetworkModel, self).__init__()
15          # KrishivB: setup the NN model and print it in the logs
16          self.model = nn.Sequential(nn.Linear(len(input_features[0][0]), 60),
17                                     nn.ReLU(),
18                                     nn.Linear(60, 1))
19          print()
20          print(self.model)
21          print()
22
23
24      def forward(self, x):
25          y_predicted = torch.sigmoid(self.model(x))
26          return y_predicted
```

Results

- **Model: Sequential(**
  (0): Linear(in_features=13, out_features=60, bias=True)
  (1): ReLU() ← Activation Function
  (2): Linear(in_features=60, out_features=1, bias=True))

1 hidden layer

- **Results (300 iterations, lr = 0.0001)**
  - optimal_p = 0.9525
  - Best True Positive (AGN Successfully Identified): 279 (290)
  - Best AGN/True Positive/Sensitivity : 0.962068965517241 4
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.9633333333333334

Results

- **Model: Sequential(**
  (0): Linear(in_features=13, out_features=60, bias=True)
  (1): ReLU()
  (2): Linear(in_features=60, out_features=1, bias=True))

- **Results (60 iterations, lr = 0.0001)**
  - optimal_p = 0.9525
  - Best True Positive (AGN Successfully Identified): 279 (290)
  - Best AGN/True Positive/Sensitivity : 0.962068965517241 4
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.963333333333334

Results

- **Model: Sequential(**
  (0): Linear(in_features=13, out_features=40, bias=True)
  (1): Tanh() ← tested with tanh() activation function also
  (2): Linear(in_features=40, out_features=1, bias=True))

- **Results (300 iterations, lr = 0.0001)**
  - optimal_p = 0.9544
  - Best True Positive (AGN Successfully Identified): 278 (290)
  - Best AGN/True Positive/Sensitivity : 0.9586206896551724
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.96

Results: DNN - 2 hidden layers

Files for DNN: 2 hidden layers

- Python Scripts
  - 4FGLReproductionBasicDNN.py
  - FGL4Dataset.py
  - DeepNeuralNetwork.py
  - utils.py
- Execution
  - python3 4FGLReproductionBasicDNN.py

2 hidden layers NN
ReLU activation function

main FermiLAT / Python / DeepNuralNetwork.py

krishivbhatia changed model names

Code Blame 55 lines (49 loc) · 2.32 KB Code 55% faster with GitHub Copilot Raw

```
1   import math
2   import torch
3   import torch.nn as nn
4
5
6   # Logistic Regression Model
7   #   https://www.youtube.com/watch?v=OGpQxIkR4ao&list=PLqnslRFeH2UrcDBWF5mfPGpqQDSta6VK4&index=8
8   # However I used 2 layers of neurons instead of 1 as it yields better results.
9   # 1 layers of neurons would just be 1/(1 + e^-(input1 * weight1 + input2 * weight2 + input3 * weight3 ... + bias))
10  class DeepNeuralNetworkModel(nn.Module):
11      def __init__(self, input_features):
12          print("(len(input_features[0][0]) = ", len(input_features[0][0]))
13          print("input_features[0][0] = ", input_features[0][0])
14          super(DeepNeuralNetworkModel, self).__init__()
15          self.model = nn.Sequential(nn.Linear(len(input_features[0][0]), 250),
16                                     nn.ReLU(),
17                                     nn.Linear(250, 40),
18                                     nn.ReLU(),
19                                     nn.Linear(40, 1))
20          print()
21          print(self.model)
22          print()
23
24
25      def forward(self, x):
26          y_predicted = torch.sigmoid(self.model(x))
27          return y_predicted
```

Results

- **Model: Sequential(**
  (0): Linear(in_features=13, out_features=250, bias=True)
  (1): ReLU() ← activation function
  (2): Linear(in_features=250, out_features=40, bias=True)
  (3): ReLU() ← activation function
  (4): Linear(in_features=40, out_features=1, bias=True))

- **Results (200 iterations, lr = 0.0001)**
  - optimal_p = 0.9475
  - Best True Positive (AGN Successfully Identified): 282 (290)
  - Best AGN/True Positive/Sensitivity : 0.9724137931034482
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.9733333333333334

Results

- **Model: Sequential(**
  (0): Linear(in_features=13, out_features=250, bias=True)
  (1): ReLU()
  (2): Linear(in_features=250, out_features=40, bias=True)
  (3): ReLU()
  (4): Linear(in_features=40, out_features=1, bias=True))

- **Results (300 iterations, lr = 0.0001)**
  - optimal_p = 0.9475
  - Best True Positive (AGN Successfully Identified): 282 (290)
  - Best AGN/True Positive/Sensitivity : 0.9724137931034482
  - Best True Negative (PSR Successfully Identified): 10 (10)
  - Best Pulsar/True Negative/Specificity : 1.0
  - Accuracy of Best P Fit : 0.9733333333333334

Questions

- Hardness ratios of LAT energy bands
  - 3FGL Catalog had many bands:
    - "Flux100_300", "Flux300_1000", "Flux1000_3000", "Flux3000_10000", "Flux10000_100000"
  - 4FGL Catalog has only one energy band column present:
    - Energy_Flux100
      - From Abdollahi 2020, definition: *Energy flux from 100MeV to 100GeV obtained by spectral fitting*
  - Where do we get other bands to calculate hardness ratios?

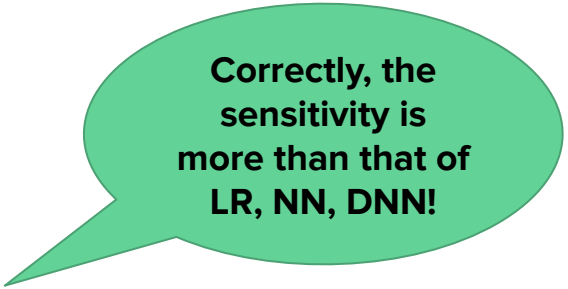
Next Steps

- Random Forest & Decision Tree implementations
  - Found Random Forest & Decision Tree implementations using PyTorch
    - https://discuss.pytorch.org/t/how-can-i-use-knn-random-forest-models-in-pytorch/42464/2
    - https://github.com/ValentinFigue/Sklearn_PyTorch
  - Test class shows how to use it:
    - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/random_forest_tests.py
    - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/binary_tree_tests.py
  - Use it to implement Random Forest & Decision Trees in PyTorch for 4FGL Catalog?

Update: 01/15/2024

Progress

- Completed Random Forest & Decision Tree implementations using PyTorch for 4FGL Catalog file
  - Random Forest
    - Highest sensitivity = 98.05749805749805
  - Decision Trees
    - Highest sensitivity = 98.13519813519814



Background

- Referenced Random Forest & Decision Tree implementations using PyTorch
  - https://github.com/ValentinFigue/Sklearn_PyTorch
- Random Forest test class shows how to use it:
  - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/random_forest_tests.py
    - random_forest = Sklearn_PyTorch.TorchRandomForestRegressor(nb_trees=10, nb_samples=2, max_depth=3)
    - random_forest.fit(train_features, train_labels)
- Decision Tree test class shows how to use it:
  - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/binary_tree_tests.py
    - binary_tree = Sklearn_PyTorch.TorchDecisionTreeClassifier(max_depth=1)
    - binary_tree.fit(train_features, train_labels)

Code Updates

- Checked in code in github for:

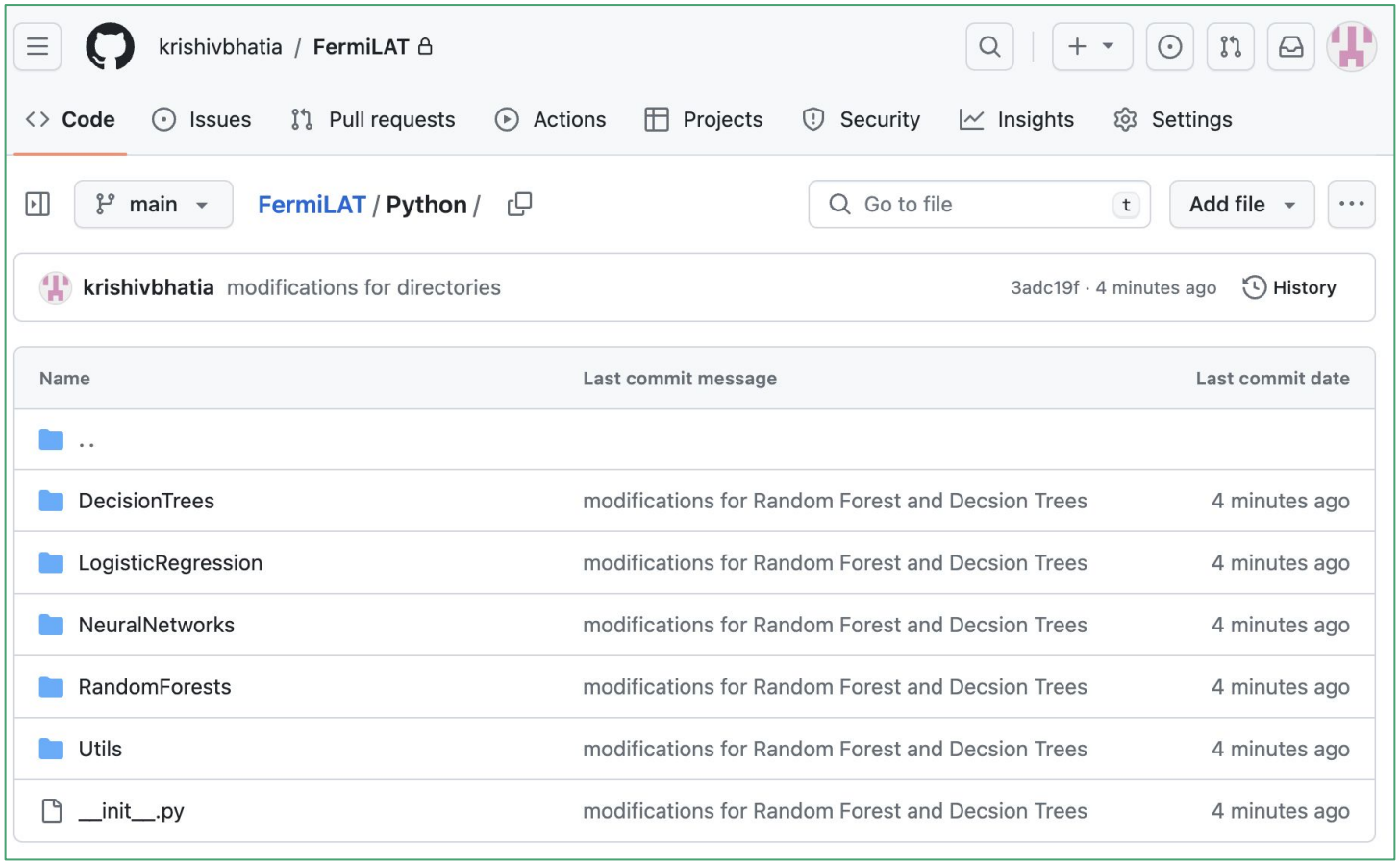
Random Forest

- 4FGLReproduction_RF.py
  - FGL4Dataset.py
  - random_forest.py
  - utils.py

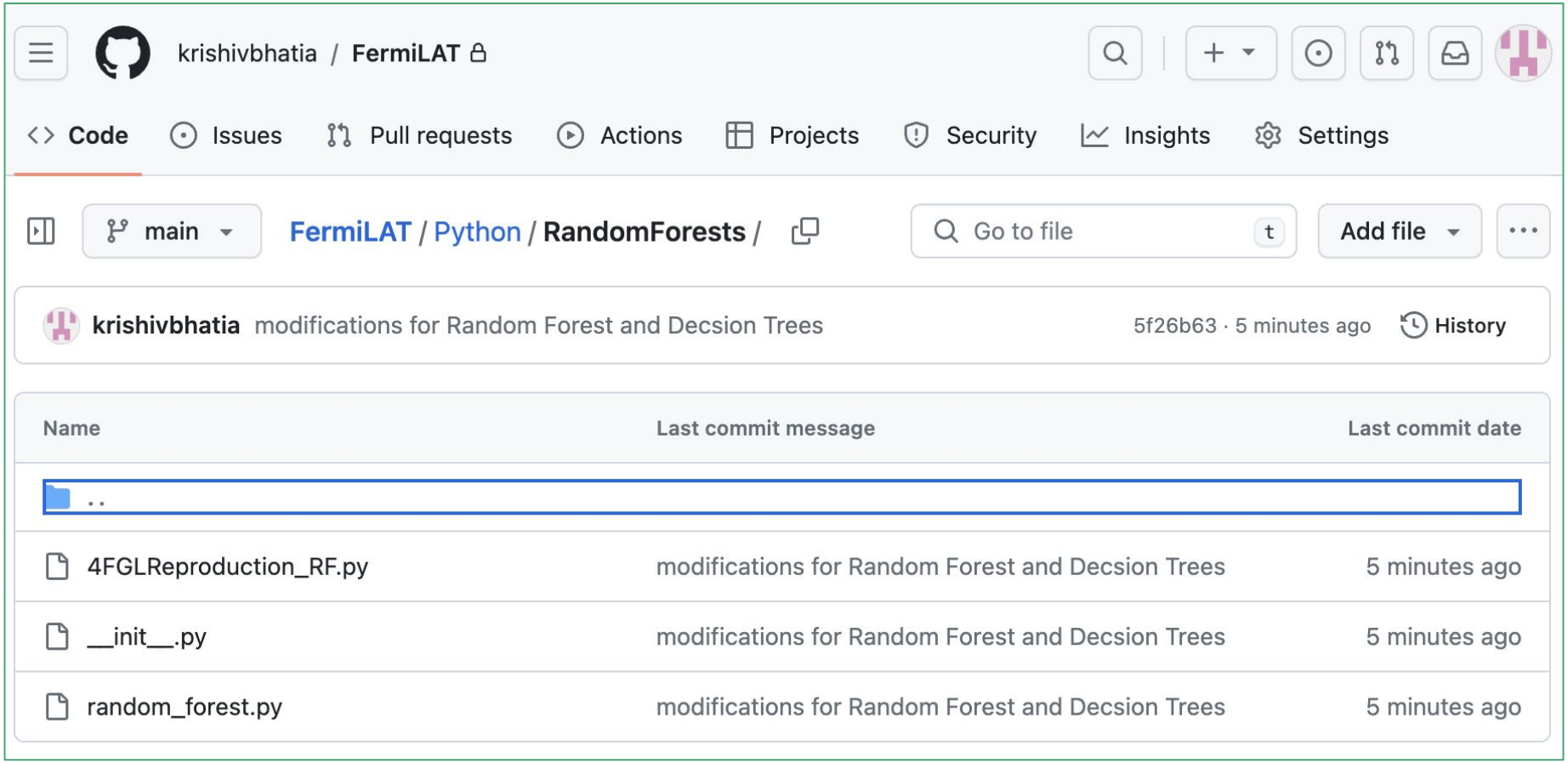
Decision Trees

- 4FGLReproduction_decision_tree.py
  - FGL4Dataset.py
  - decision_tree.py
  - decision_node.py
  - utils.py

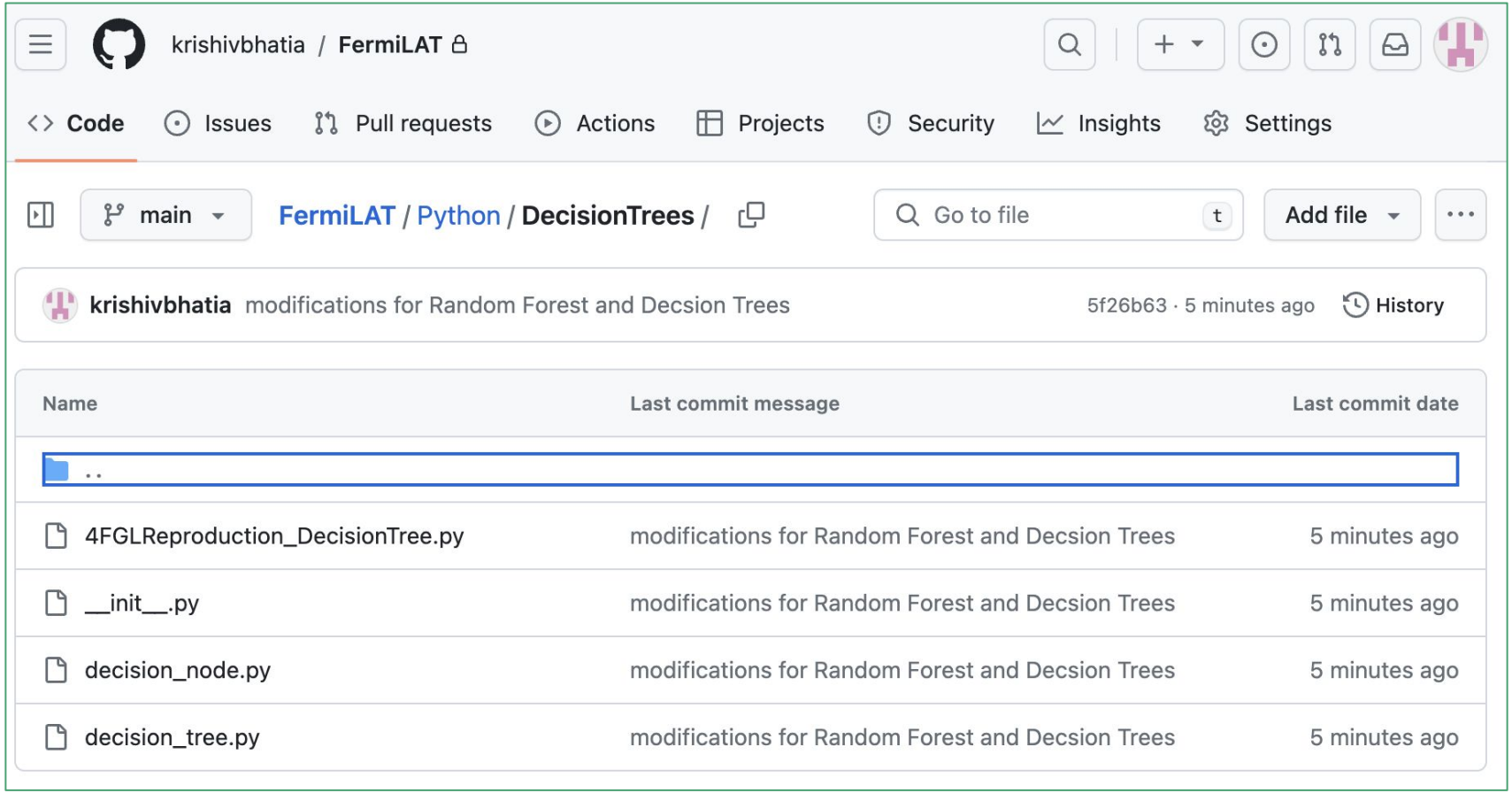
Github Repo - Python package



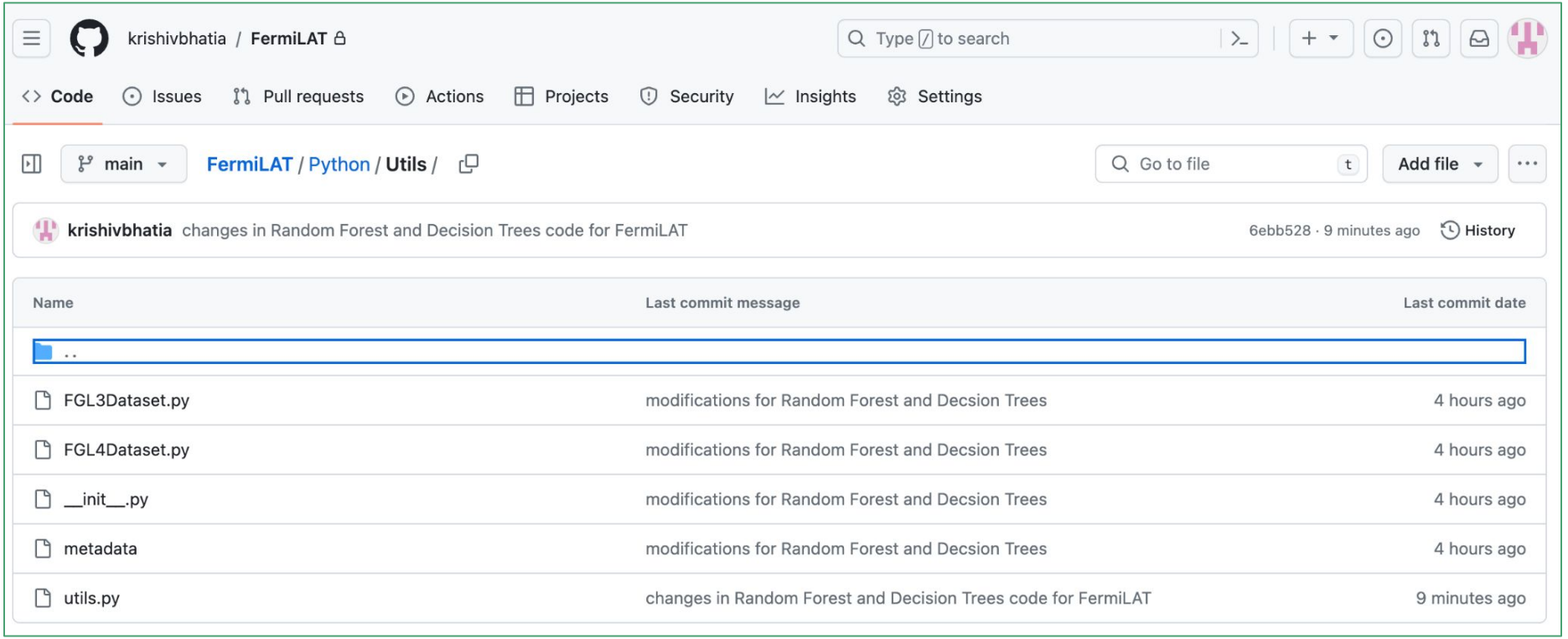
Github Repo - Random Forests package



Github Repo - Decision Trees package



Github Repo - Utils package



Code Details

- Method of creating classifier and invoking fit method:

Transforms dataset into separate feature & labels arrays (next slide)

Random Forest

4FGLReproduction_RF.py

```
# Krishiv Bhatia: Invoke TorchRandomForestClassifier
random_forest = TorchRandomForestClassifier( nb_trees: 200,  nb_samples: 2500,  max_depth: 15)
print("random forest = ", random_forest.nb_trees, random_forest.nb_samples, random_forest.max_depth)

# Krishiv Bhatia: split train dataset into features and labels
train_features, train_labels = dataset_to_features_labels(train_dataset)
random_forest.fit(torch.FloatTensor(train_features), torch.LongTensor(train_labels))
```

Decision Trees

4FGLReproduction_decision_tree.py

```
# Krishiv Bhatia: Invoke TorchDecisionTreeClassifier
decision_tree = TorchDecisionTreeClassifier(20)
print("TorchDecisionTreeClassifier = ", decision_tree.max_depth)

# Krishiv Bhatia: split train dataset into features and labels
train_features, train_labels = dataset_to_features_labels(train_dataset)
decision_tree.fit(torch.FloatTensor(train_features), torch.LongTensor(train_labels))
```

Transformations

- Transformed the tensor feature and labels array into regular arrays

main ▾ FermiLAT / Python / Utils / utils.py

Blame 150 lines (126 loc) · 4.75 KB Code 55% faster with GitHub Copilot

```
def mean(values):
    Mean function.
    """
    m = 0.0
    for value in values:
        m = m + value/len(values)
    return m


def dataset_to_features_labels(dataset):
    # Krishiv Bhatia: train or test dataset is of the form:
    #    tensor array(train/test features), tensor array (output label)
    # or like this:
    #    tensor array(feature 0, feature 1, ... feature 12), tensor array(output label)
    # You can print it like this:
    #     for i in train_dataset:
    #         print(i)
    # An example:
    #    (tensor([  2.1602,   1.8308,   1.6083,  13.2215, -32.9276, -32.4171, -29.4589,
    #             -32.3813, -24.3150,   0.4812,   0.0000,   2.7429,   5.4350]), tensor([1.]))
    #
    # Convert tensor feature array in dataset to numpy feature array
    train_features = ([((i[0]).detach().numpy()) for i in dataset])
    # print("train_features = ", train_features)
    # Convert tensor label array in dataset to regular array
    train_labels = ([torch.LongTensor.item(i[1]) for i in dataset])
    # print("train_labels = ", train_labels)
    return train_features, train_labels
```

i[0] gets tensor feature array
i[1] gets tensor label array

References

- https://stackoverflow.com/questions/49768306/pytorch-tensor-to-numpy-array
- https://stackoverflow.com/questions/54268029/how-to-convert-a-pytorch-tensor-into-a-numpy-array
- https://stackoverflow.com/questions/67711752/how-to-extract-tensors-to-numpy-arrays-or-lists-from-a-larger-pytorch-tensor
- https://stackoverflow.com/questions/47588682/how-to-cast-a-1-d-inttensor-to-int-in-pytorch
- https://stackoverflow.com/questions/57727372/how-do-i-get-the-value-of-a-tensor-in-pytorch

Random Forest Results

- `random_forest = TorchRandomForestClassifier(50, 2000, 10)`
  - `correct =  1262/1287`
  - `sensitivity = 98.05749805749805`
- `random_forest = TorchRandomForestClassifier(200, 2000, 15)`
  - `correct =  1258/1287`
  - `sensitivity = 97.74669774669775`
- `random_forest = TorchRandomForestClassifier(200, 1500, 15)`
  - `correct =  1262/1287`
  - `sensitivity = 98.05749805749805`

Random Forest Results

- `random_forest = TorchRandomForestClassifier(15, 500, 10)`
  - `correct =  1261/1287`
  - `sensitivity = 97.97979797979798`
- `random_forest = TorchRandomForestClassifier(100, 500, 10)`
  - `correct =  1255/1287`
  - `sensitivity = 97.51359751359752`
- `random_forest = TorchRandomForestClassifier(50, 1000, 15)`
  - `correct =  1258/1287`
  - `sensitivity = 97.74669774669775`
- `random_forest = TorchRandomForestClassifier(50, 1500, 10)`
  - `correct =  1259`
  - `sensitivity = 97.82439782439782`

Decision Tree Results

- decision_tree = TorchDecisionTreeClassifier(10)
  - correct = 1263
  - sensitivity = 98.13519813519814
- decision_tree = TorchDecisionTreeClassifier(20)
  - correct =
  - sensitivity =
- decision_tree = TorchDecisionTreeClassifier(30)
  - correct = 1263
  - sensitivity = 98.13519813519814
- decision_tree = TorchDecisionTreeClassifier(80)
  - correct = 1263
  - sensitivity = 98.13519813519814

Update: 01/22/2024

Progress

- Completed the problem of Energy Band Level Hardness ratios
- Column: Flux_Band
- Printed it out. Numpy array with 8 numbers
    - source[flux_col] = [1.318705452994784e-12, 3.9183367661621 56e-10, 3.155801175935835e-10, 1.045405220723161 2e-10, 2.0413597895396762e-11, 6.700519479541089e-12, 1.909341825270805e-12, 2.2682772056883754e-16]

Transformation

- In FGL4Dataset.py
- Converted numpy array to list
  - `flux_list = res = source[flux_col].tolist()`
- Calculated hardness ratios

```
# Calculate hardness ratios (also considered in the paper)
for index in range(1, len(flux_list)):
   unit.append((flux_list[index] - flux_list[index-1] + 0.0) /
               (flux_list[index] + flux_list[index-1] + 0.0))
xdata.append(unit)
```

Calculates hardness ratios from one band index to next

Logistic Regression

- Model:
  - **Linear(in_features=20, out_features=1, bias=True)**
- Iterations = 30/100
  - Best P Threshold Value : 0.972
  - Score : 1.0586206896551724
  - Best Score : 1.9655172413793105
  - Best True Positive (AGN Successfully Identified): 280
  - AGN count : 290
  - Best AGN/True Positive/Sensitivity : 0.9655172413793104
  - Best True Negative (PSR Successfully Identified): 10
  - PSR count : 10
  - Best Pulsar/True Negative/Specificity : 1.0
  - **Accuracy of Best P Fit : 0.9666666666666667**
  - Best correct : 290
  - No of samples : 300

Neural Networks

- Model:
  - **Sequential(**
    (0): Linear(in_features=20, out_features=60, bias=True)
    (1): ReLU()
    (2): Linear(in_features=60, out_features=1, bias=True)
)
- Iterations = 30
  - Best P Threshold Value : 0.976
  - Score : 1.0551724137931036
  - Best Score : 1.9586206896551723
  - Best True Positive (AGN Successfully Identified): 278
  - AGN count : 290
  - Best AGN/True Positive/Sensitivity : 0.9586206896551724
  - Best True Negative (PSR Successfully Identified): 10
  - PSR count : 10
  - Best Pulsar/True Negative/Specificity : 1.0
  - **Accuracy of Best P Fit : 0.96**
  - Best correct : 288
  - No of samples : 300

Deep Neural Networks

- Model:
  - **Sequential(**
    (0): Linear(in_features=20, out_features=250, bias=True)
    (1): ReLU()
    (2): Linear(in_features=250, out_features=50, bias=True)
    (3): ReLU()
    (4): Linear(in_features=50, out_features=1, bias=True)
)
- Results
  - Best P Threshold Value : 0.971
  - Score : 1.0586206896551724
  - Best Score : 1.9724137931034482
  - Best True Positive (AGN Successfully Identified): 282
  - AGN count : 290
  - Best AGN/True Positive/Sensitivity : 0.9724137931034482
  - Best True Negative (PSR Successfully Identified): 10
  - PSR count : 10
  - Best Pulsar/True Negative/Specificity : 1.0
  - **Accuracy of Best P Fit : 0.9733333333333334**
  - Best correct : 292
  - No of samples : 300

Deep Neural Networks

- DNN
- Model:
  - **Sequential(**
    (0): Linear(in_features=20, out_features=250, bias=True)
    (1): ReLU()
    (2): Linear(in_features=250, out_features=40, bias=True)
    (3): ReLU()
    (4): Linear(in_features=40, out_features=1, bias=True)
)
  - Best P Threshold Value : 0.967
  - Score : 1.0517241379310345
  - Best Score : 1.9689655172413794
  - Best True Positive (AGN Successfully Identified): 281
  - AGN count : 290
  - Best AGN/True Positive/Sensitivity : 0.9689655172413794
  - Best True Negative (PSR Successfully Identified): 10
  - PSR count : 10
  - Best Pulsar/True Negative/Specificity : 1.0
  - **Accuracy of Best P Fit : 0.97**
  - Best correct : 291
  - No of samples : 300

Random Forests

- Model:
  - **# Trees: 200**
  - **# Samples: 2500**
  - **Depth: 15**
- Results
  - Correct = 1258
  - Toital = 1286
  - Sensitivity = 97.82439782439782

Random Forests

- Model:
  - **# Trees: 200**
  - **# Samples: 1500**
  - **Depth: 15**
- Results
  - Correct = 1258
  - Toital = 1286
  - Sensitivity = 97.74669774669775

Random Forests

- Model:
  - **# Trees: 200**
  - **# Samples: 500**
  - **Depth: 15**
- Results
  - Correct = 1254
  - Toital = 1286
  - Sensitivity = 97.43589743589743

Decision Trees

- Decision Trees
- Model:
  - **Depth: 20**
  - Correct = 1259
  - Toital = 1286
  - Sensitivity = 97.82439782439782

Next Steps

- Could you kindly clarify what needs to be done with the following tasks
  - associated/unassociated sources
  - AGN types